

Efficient Multimodal Serving via Module Multiplexing

Zicong Hong[¶] Yuyan Chen[†] Haoyue Zhang[¶] Peng Li^{\$}

Wuhui Chen[†] Song Guo[¶] Xiaowei Shen^ζ

[¶]The Hong Kong University of Science and Technology [†]Sun Yat-sen University

^{\$}Xi'an Jiaotong University ^ζMetaX

congcong@ust.hk, chenyy536@mail2.sysu.edu.cn, hzhangex@connect.ust.hk, pengli@xjtu.edu.cn

chenwuh@mail.sysu.edu.cn, songguo@cse.ust.hk, xiaowei.shen@metax-tech.com

Abstract

Multimodal learning enables models to process and reason over diverse information sources, unlocking human-like perceptual and cognitive capabilities. As such models gain adoption, efficiently serving them on GPUs has become increasingly important. However, the modular architecture of multimodal models poses significant challenges to existing unimodal serving systems, which treat models as monolithic and overlook inter-module heterogeneity. This results in severe GPU underutilization. To address this, we propose EEVEE, a multimodal serving system based on a new scheduling paradigm we call *module multiplexing*. Unlike prior approaches that execute all modules sequentially with uniform batch sizes, EEVEE schedules modality-specific modules concurrently on the same GPU with independently tuned batching and resource allocation. This design enables fine-grained GPU sharing, boosting intra-GPU parallelism and improving request-level throughput. We implement a prototype of EEVEE and evaluate it on several representative multimodal models (e.g., CLIP, BLIP, LLaVA, InternVL). Our results show that EEVEE significantly outperforms state-of-the-art serving systems in both throughput and GPU utilization.

CCS Concepts: • Computing methodologies → Machine learning.

Keywords: Multimodal model, GPU multiplexing, efficient serving system

ACM Reference Format:

Zicong Hong[¶] Yuyan Chen[†] Haoyue Zhang[¶] Peng Li^{\$}, Wuhui Chen[†] Song Guo[¶] Xiaowei Shen^ζ. 2026. Efficient Multimodal Serving via Module Multiplexing. In *European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland Uk. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3767295.3769389>



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, April 27–30, 2026, Edinburgh, Scotland Uk

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04...\$15.00

<https://doi.org/10.1145/3767295.3769389>

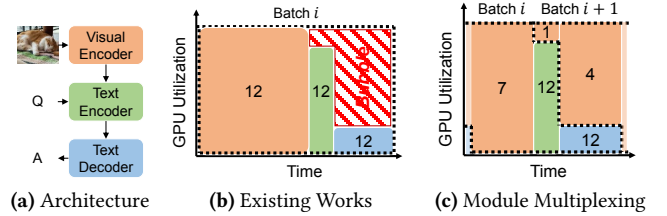


Figure 1. Comparison between the existing multimodal model serving scheme and our EEVEE (i.e., module multiplexing). Each number denotes a module's batch size.

1 Introduction

Multimodal learning is widely regarded as a transformative step toward artificial general intelligence, aiming to develop models capable of ingesting, interpreting, and reasoning over diverse information sources. Driven by advances in transformer architectures, a range of powerful pre-trained multimodal models have emerged, including CLIP [44], BLIP [32], BLIP-2 [31], and LLaVA [35], which have fueled a broad spectrum of downstream applications [3, 22, 33, 54, 58].

As these models become increasingly prevalent, the need for efficient multimodal model serving—achieving low latency and high resource utilization—has become critical. While prior work has made significant progress in developing model serving systems, such as TF-Serving [40], Clipper [12], TurboTransformers [18], and Triton [39], these systems are primarily designed for unimodal models and fall short when serving complex multimodal architectures.

These systems typically treat a model as a monolithic computation graph, executing each layer sequentially with a uniform batch size. To improve GPU utilization, they scale up the batch size until the inference latency violates the Service-Level Objective (SLO). However, such strategies become inefficient when applied to multimodal models. For example, our preliminary experiments (Figure 4 in § 3) show that GPU utilization drops below 10% for over 50% of the time when serving BLIP on a Visual Question Answering (VQA) task using NVIDIA Triton.

This inefficiency stems from a fundamental mismatch: existing unimodal serving systems fail to account for the intra-model heterogeneity introduced by modality-specific

modules with varying computational characteristics. A typical multimodal model comprises multiple such modules, each implemented as a separate transformer tailored to process a specific modality. These modules often adopt different architectures and exhibit vastly different performance profiles [28]. For instance, as shown in Figure 1a, BLIP for VQA consists of three modules: a text encoder, a visual encoder, and a text decoder (see § 2.1). As shown in Figure 1b, GPU utilization fluctuates sharply between modules: the visual encoder is highly compute-intensive and often constrained by insufficient GPU resources, while the text encoder and decoder underutilize the GPU, introducing substantial GPU bubbles. This imbalance leads to poor intra-GPU utilization and degraded overall throughput.

Besides the architectural differences among modules discussed above, multimodal models exhibit two other forms of inter-module heterogeneity that further exacerbate GPU underutilization under the existing unimodal serving systems.

First, there is an imbalance in input patterns across modalities, resulting in uneven workloads. Computing each module sequentially with the same batch size can lead to GPU waste for modules with lighter workloads. For example, in a VQA application, a user's conversation may contain several questions referring to the same image, targeting different regions or tasks (e.g., fine-grained recognition, activity detection, or knowledge-based reasoning) [1]. In such cases, the workload on the visual encoder remains relatively low compared to that of the text encoder. Similar behavior is observed in other modalities such as videos [52] and tables [6].

Second, preprocessing demands vary widely across modalities. Preprocessing is essential to ensure data consistency, normalization, and effective feature extraction, and typically involves CPU computation and I/O operations. Visual inputs—such as images and videos—require significantly more preprocessing time than text due to their higher dimensionality and noise (see § 3). When preprocessing and inference are performed sequentially, the GPU may remain idle while waiting for preprocessing to complete.

The interplay of these three forms of inter-module heterogeneity motivates us to propose a new serving paradigm called *module multiplexing*. As illustrated in Figure 1c, we move away from treating the multimodal model as a monolithic unit for scheduling and execution. Instead, we consider modality-specific modules as independently schedulable units, each with its own batch size and execution timeline on the same GPU. This enables fine-grained intra-GPU parallelism, improving GPU utilization and throughput.

Despite its potential, module multiplexing introduces several challenges. First, determining an effective scheduling strategy is non-trivial, as modality-specific modules must not only compete for shared GPU resources but also cooperate to complete each inference request. Over-allocating resources to one module may accelerate its execution, but can leave other modules starved, ultimately shifting the bottleneck

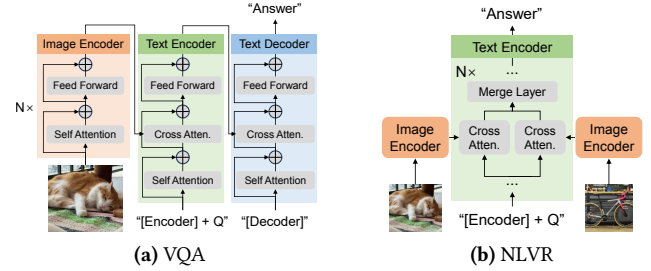


Figure 2. The architecture of BLIP.

and reducing overall throughput. Second, unlike sequential execution, where intermediate outputs are immediately forwarded to the next module, module multiplexing disrupts this execution order. Consequently, the outputs of upstream modules must be temporarily cached until their downstream consumers are scheduled, introducing memory and I/O overhead that must be carefully managed by the serving system.

To address these challenges, we introduce EEVEE, an efficient multimodal model serving system based on module multiplexing. EEVEE analyzes the characteristics of modality-specific modules and available GPU resources to schedule and cache their computations independently. It orchestrates fine-grained resource allocation across modules to improve throughput while maintaining system efficiency.

The key contributions of this paper are as follows:

- We present an in-depth empirical analysis of the performance bottlenecks in existing unimodal serving systems when applied to multimodal models. Based on this, we identify three core sources of inefficiency and motivate the design of module multiplexing.
- To mitigate the additional system overhead caused by module multiplexing, we propose a module caching control design that supports efficient management (including identification, caching, and reusing) of module intermediate inference results.
- To strike a balance between resource competition and inference cooperation between modules, we propose a new heuristic algorithm that operates within a constrained search space to orchestrate computations and allocate resources between modules.
- We implement a prototype of EEVEE and evaluate it in CLIP, BLIP, LLaVA, and InternVL in various multimodal tasks. The results show that it increases the throughput by 157% and reduces the latency by about 90%, compared to state-of-the-art serving systems.

2 Background

2.1 Multimodal Models

Multimodal Transformer Models (MTMs). An MTM consists of several *modules*, each of which is a transformer model. We use the well-established MTM, BLIP [32], as an example.

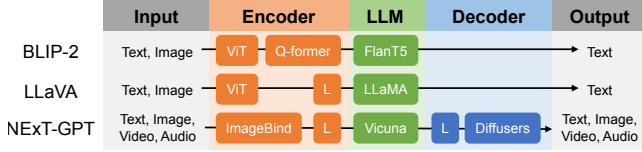


Figure 3. Comparison of the overview schematic of recent proposed MLLMs. L represents linear projection layers.

The overall architecture, depicted in Figure 2a, consists of three modules. The vision module uses ViT [16] as the image encoder to extract visual features. The two text modules are based on BERT [14]: one is a text encoder that extracts text features, and the other is a text decoder that generates text output. Each module includes a set of self-attention and feedforward networks to capture contextual relationships within its respective modality. Some modules incorporate self-attention variants (e.g., early summation, hierarchical attention, cross-attention) to enable cross-modal interactions. In BLIP, the text encoder and text decoder employ cross-attention to integrate visual and textual information. As shown in Figure 2b, another task, Natural Language Visual Reasoning (NLVR), determines if a sentence corresponds to a description of a pair of images [48].

Multimodal Large Language Models (MLLMs). The remarkable capabilities of Large Language Models (LLMs) have inspired efforts to integrate them with existing multimodal models [53]. An MLLM can be conceptualized as a combination of several modules: multiple encoders for different input modalities, a pre-trained LLM, and multiple decoders for outputting various modalities. As shown in Figure 3, BLIP-2 [31] uses ViT [16] alongside a lightweight transformer called Q-Former as its visual encoder, with Flan-T5 [11] or OPT [56] as its LLM. LLaVA replaces the Q-Former structure with a single linear layer [35]. NExT-GPT [51] utilizes ImageBind [21], a versatile six-modality encoder, and various diffusion models for different modal generations, including Stable Diffusion (SD) for image synthesis, Zeroscope for video synthesis, and AudioLDM for audio synthesis.

MTMs vs MLLMs. The emergence of MLLMs does not mean that traditional MTMs should be discarded. MTMs require fewer resources and achieve relatively good performance across a wide range of multimodal tasks. As such, they are well suited for edge scenarios, and this paper considers both MTMs and MLLMs. As many MTMs and MLLMs follow similar architectures such as BLIP and BLIP-2 respectively, our paper focuses mainly on their serving.

2.2 Model Serving

Existing serving systems handle inference requests using various accelerators (e.g. GPUs). Although GPU-based systems offer low latency for inference, high utilisation of GPU computing resources requires the following technologies.

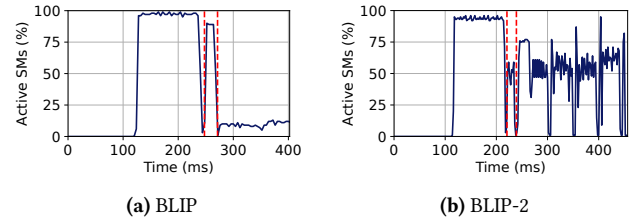


Figure 4. Percentage of active SMs in the GPU while processing a batch of requests in Triton.

Batching. It combines multiple model inference requests into a single batch to improve GPU utilisation. This takes advantage of the parallel processing capabilities of the GPU to process multiple inputs simultaneously and efficiently reduce idle time. Effective batching requires careful consideration of batch size, latency constraints and workload characteristics. The challenge is to balance latency and throughput, as overly large batches can result in unacceptable delays for end users.

GPU Multiplexing. It allows multiple workloads to share GPU. This involves leveraging the GPU’s Streaming Multiprocessors (SMs), the units executing parallel tasks. Temporal sharing schedules different tasks simultaneously, maintaining consistent GPU activity without resource contention. Spatial sharing enables multiple tasks to run concurrently by distributing workloads across SMs. Multi-Process Service (MPS) [37] and multi-instance GPU (MIG) [36] enables simultaneous execution of multiple models. This paper is based on MPS because MIG provides only rigid partitioning, leading to GPU overprovisioning and underutilization, and is only available in the latest generation of GPUs.

3 Motivations

3.1 Observations

Finding 1. Low GPU Utilization: Using existing unimodal serving systems to serve multimodal models leads to significant GPU under-utilization.

We begin by evaluating the GPU utilization (i.e., the percentage of active SMs) when serving multimodal models with existing systems. We use BLIP and BLIP-2 on the VQA task, deployed in NVIDIA Triton on a GeForce RTX 3090 GPU. Implementation details are provided in § 8.1. The batch size is set to 8. Figure 4 illustrates the GPU utilization throughout the processing of a batch of requests. While the GPU occasionally reaches full utilization, it remains under-utilized for most of the time. As shown in Figure 4a, the inference is divided into three phases, marked by two red dotted lines: visual encoder, text encoder, and text decoder. The GPU reaches near-full utilization only during the latter half of the first phase. Similarly, Figure 4b shows the three phases

Table 1. Details of modules in BLIP and BLIP-2.

	Module	Model	Params
BLIP	Visual Encoder	ViT-B/16	86M
	Text Encoder	BERT _{base}	110M
	Text Decoder		
BLIP-2	Visual Encoder	ViT-g/14	1B
	Q-Former	BERT _{base}	188M
	LLM	FlanT5 _{XL}	2.8B

for BLIP-2: visual encoder, Q-Former, and LLM. We identify three main causes for this low utilization, elaborated below.

Finding 2. Inference Heterogeneity: *The computational complexity of modules in a multimodal model varies significantly due to differences in architecture and modality-specific processing mechanisms.*

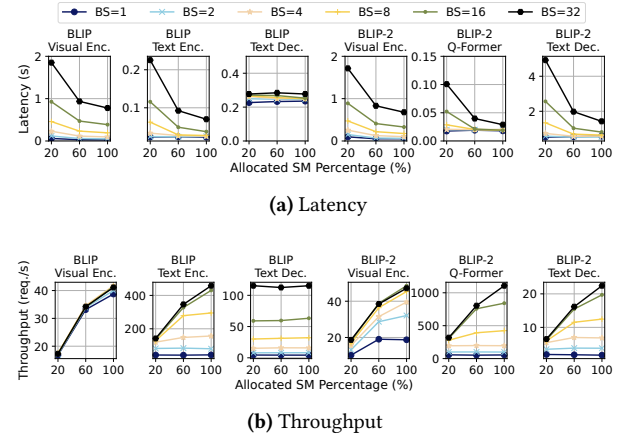
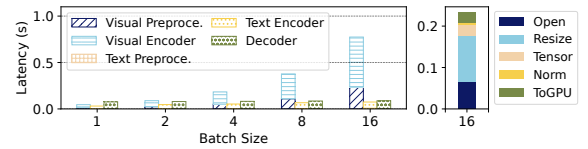
As described in § 2.1, a key distinction between unimodal and multimodal models is that the latter consist of multiple modules with distinct architectures and computation characteristics, each optimized for a specific modality. These modules exhibit divergent GPU resource usage patterns.

To illustrate this heterogeneity, Table 1 shows the parameter settings of BLIP and BLIP-2 on the VQA task. Their performance varies drastically. As shown in Figure 5a, the visual encoder in BLIP exhibits up to 8× the latency of the text encoder. In addition, their throughput responds differently to batch size scaling. Figure 5b shows that the text encoder’s throughput improves substantially when increasing the batch size from 1 to 8, indicating that small batches fail to fully utilize GPU resources. In contrast, the visual encoder—being more compute-intensive—achieves near-maximum utilization even at small batch sizes. The text encoder reaches saturation around a batch size of 8, while the text decoder continues to benefit from batching beyond 32.

According to **Finding 2**, applying the same batch size and allocating 100% of SMs to each module—following the convention of unimodal model serving—can result in low GPU utilization. This motivates a shift toward treating each module as an independently schedulable unit, allowing customized batching and resource allocation that better align with its computational characteristics.

Finding 3. Preprocessing Overhead: *Multimodal model serving relies on CPU resources for data preprocessing. The CPU workload is heterogeneous across modalities, as different input types require distinct preprocessing operations.*

In unimodal language-only models, the input is text, which undergoes lightweight preprocessing such as tokenization—i.e.,

**Figure 5.** Performance of different modules in BLIP and BLIP-2 for VQA with different batch sizes and SM percentages.**Figure 6.** Input preprocessing and inference latency of different modules in BLIP for VQA (left). Latency of visual preprocessing operations for a batch size of 16 (right).

converting text into token sequences, embedding them numerically, and assembling them into tensors. Since tokenization introduces negligible latency, unimodal serving typically performs preprocessing and inference serially without noticeable performance degradation. In contrast, multimodal models handle inputs from diverse modalities, each requiring more complex preprocessing beyond tokenization. For example, image inputs may need resizing, normalization, and color space conversion; video inputs may require frame extraction; and audio inputs often involve discrete cosine transforms or resampling [9].

The latency of such operations can account for a significant portion of overall inference time. For instance, in BLIP, image preprocessing involves image loading, resizing, tensor conversion, normalization, and data transfer to GPU memory. As shown in Figure 6, this preprocessing step can take up to 40% of the visual encoder’s computation time.

According to **Finding 3**, executing preprocessing and inference serially—especially in multimodal settings—leaves the GPU idle during preprocessing, resulting in poor hardware utilization. This motivates a finer-grained view of execution, where each module is further divided into preprocessing and inference stages that can be scheduled independently. Such stage-level parallelism helps reduce GPU idle time caused by input preparation delays.

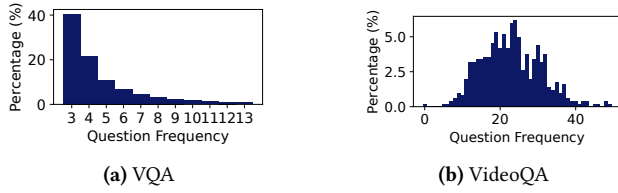


Figure 7. The distribution of query frequency for each image or video in the test set of two multimodal tasks.

Finding 4. Input Imbalance: In a multimodal model serving system, the distribution of inputs from different modalities may be imbalanced.

In multimodal serving, user interactions often include multiple queries referring to previous input modalities. For instance, in the VQA task, users may ask multiple questions targeting specific areas or tasks (e.g., fine-grained recognition, activity recognition, or knowledge-based reasoning) of an image [1]. For example, given an image, a user might ask: “Here’s a photo of my garden. Can you identify the plants?” Subsequently, they might follow up with: “In the photo, which of these plants needs the most sunlight?” As shown in Figure 7a, the VQA2 dataset [20] reveals that each image is associated with at least three questions—and in some cases, up to 246. Similarly, in the VideoQA task, as illustrated in Figure 7b, each video in the MSVD-QA dataset [52] averages around 20 questions. Comparable patterns are also found in tasks involving tables and other modalities [6].

According to **Finding 4**, not all modality-specific modules are needed for every request. Recomputing all modules—regardless of whether their outputs are required—wastes compute resources and increases latency. This further reinforces the need to treat modules as independently schedulable units, enabling fine-grained execution paths that match the actual query requirements.

3.2 Strawman

Existing unimodal serving systems typically treat models as monolithic entities, executing all layers sequentially with uniform batch sizes and identical GPU resource allocation across requests. As discussed in § 3.1, this approach is ill-suited for multimodal serving due to the inherent heterogeneity in module computation, input patterns, and preprocessing requirements. The result is consistently low GPU utilization. To address this limitation, we propose a novel serving scheme called *module multiplexing* tailored for multimodal models. A preliminary strawman system is outlined below.

Design. 1) *Module-level scheduling*: Each module is scheduled independently, enabling parallel execution with varying batch sizes based on its computational characteristics. 2) *Stage-level parallelism*: Preprocessing and inference within

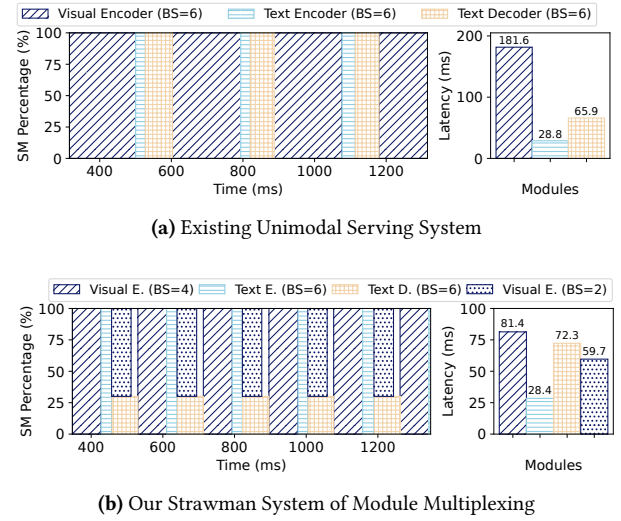


Figure 8. Pipeline (left) and latency (right) of modules in BLIP for VQA. The throughput for the existing system is 22.2 requests/s, and for module multiplexing is 34.4 requests/s.

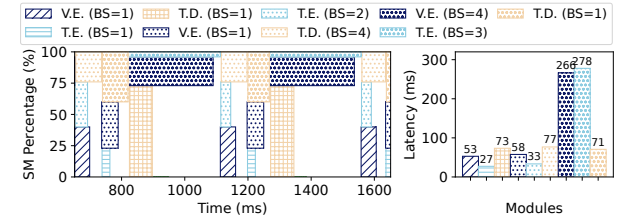


Figure 9. Pipeline (left) and latency (right) of the module multiplexing scheme with a bad strategy different from that in Figure 8b. The throughput is 14.1 requests/s.

a module are treated as separate stages that can be executed concurrently to improve pipeline utilization. 3) *Request-aware reuse*: Not all modules are required for every request; modules whose outputs are shared across requests can cache and reuse results to avoid redundant computation.

Preliminary Experiment. We implement a prototype of the strawman design; implementation details are provided in § 7. Both the strawman and the unimodal-style baseline are evaluated with a batch size of 6. In BLIP for VQA, we apply a specific multiplexing strategy illustrated in Figure 8b. In each batch, the inference for the visual encoder on two requests is shifted to run concurrently with the text decoder of the previous batch and is allocated 70% of the SMs. Preprocessing and inference are executed in parallel, allowing GPU computation to begin without waiting for CPU-side preprocessing to complete. Finally, request-aware reuse is applied by caching the outputs of the visual encoder to avoid recomputation for repeated image inputs. Compared to the baseline (Figure 8a), our prototype achieves a 54.95% improvement in throughput.

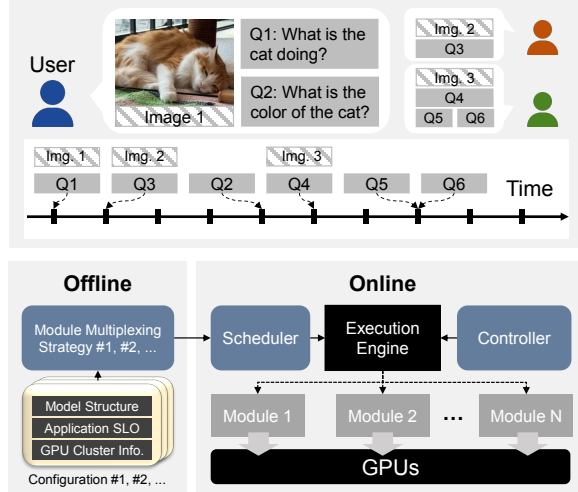


Figure 10. Architecture overview of EEVEE.

Challenges. Despite these initial gains, module multiplexing introduces several system-level challenges. First, designing an effective multiplexing strategy is non-trivial, as modules not only compete for limited GPU resources but must also coordinate to fulfill each request. As shown in Figure 9, an alternative scheduling plan led to lower throughput than the baseline due to excessive resource contention. Second, fine-grained scheduling introduces overhead. In particular, request-aware reuse requires identifying, caching, and managing intermediate module outputs, which places pressure on memory and I/O subsystems. These challenges motivate the design of a dedicated system for efficient multimodal serving: EEVEE, introduced in the next section.

4 System Overview

The overall architecture of EEVEE, illustrated in Figure 10, is designed to fully realize the benefits of module multiplexing. EEVEE operates on one or more GPUs to serve multimodal models and handle task requests from multiple users, including both context-sensitive and context-free queries involving visual inputs. EEVEE consists of two main components: a controller (§ 5) and a scheduler (§ 6). On each GPU, multiple processes are launched, each dedicated to a specific module of the multimodal model and configured with individualized batch sizes and SM allocations. GPU multiplexing enables these module processes to execute concurrently, enabling fine-grained scheduling and improved resource utilization.

To address the temporal decoupling introduced by module multiplexing—where downstream modules may not immediately follow upstream ones—a controller is deployed on each GPU. It maintains both an active cache for ongoing requests and a historical cache for completed ones. The controller manages the reuse of cached module outputs while balancing the compute demands of new requests with the I/O overhead associated with cache access and eviction.

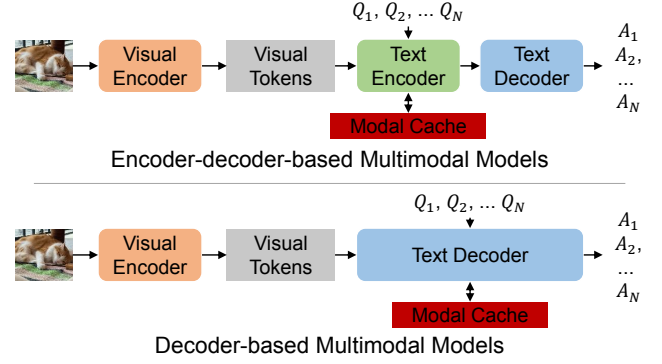


Figure 11. Modal cache (highlighted in red) for serving different multimodal models in context-insensitive scenarios.

To mitigate inter-module resource contention and preserve cooperative inference behavior caused by module multiplexing, a global scheduler coordinates module execution across the system. During the offline phase, it generates a set of multiplexing strategies, each tailored to a specific deployment configuration defined by the type of model, application requirements, and GPU hardware. At runtime, the scheduler selects the appropriate strategy based on the active deployment configuration and applies it to coordinate the scheduling of module processes on each GPU, ensuring balanced and efficient execution under dynamic workloads.

5 Module Caching Control

A fundamental consequence of module multiplexing is that the execution of modules no longer follows a strict sequential order. Unlike traditional serving systems—where intermediate outputs are immediately forwarded to the next module—module multiplexing introduces temporal gaps between upstream and downstream modules. As a result, the outputs of earlier modules must be temporarily cached until their consumers are scheduled. This introduces non-trivial memory and I/O overhead that must be carefully managed.

To support this, EEVEE equips each GPU with a controller that manages intermediate results at module granularity. The controller identifies reusable outputs, caches them across requests, and orchestrates their efficient retrieval when needed. In addition to enabling asynchronous execution, this design also supports reuse of shared computations—such as avoiding redundant visual encoding for repeated inputs—thus improving both efficiency and throughput.

5.1 Module Computation Caching

We define the intermediate outputs of a multimodal model that are reusable across multi-turn interactions as the *modal cache*. The design of the modal cache depends on the model's input type, internal structure, and inter-module dependencies. While we take vision-language models as an example in this section, the approach generalizes to other modalities. In

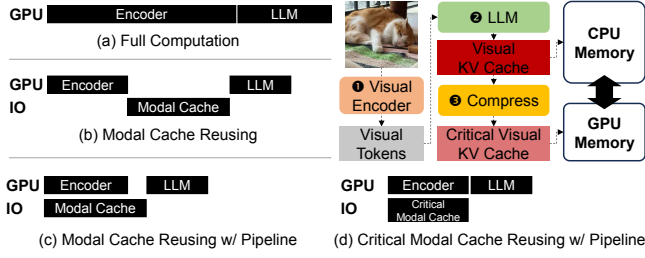


Figure 12. Illustration of modal cache management.

vision-related queries, the construction of the modal cache depends on whether the interaction is context-sensitive. In context-sensitive cases, the cache corresponds to the standard KV cache used in language models. In context-insensitive cases, however, the modal cache structure differs. As illustrated in Figure 11, we categorize vision-language models into two types: 1) Encoder-Decoder Models: For MTMs and MLLMs such as BLIP and BLIP-2 with FlanT5, the modal cache stores the KV pairs of visual tokens consumed by the cross-attention layers in the text encoder. 2) Decoder-Only Models: For MLLMs such as BLIP-2 with OPT and LLaVA, the modal cache consists of the KV pairs of visual tokens consumed by the decoder.

5.2 Modal Cache Reusing

As shown in Figure 12(a), when processing new requests that initiate a multimodal conversation, the system computes all required modules, and the controller stores the resulting modal cache in GPU memory. If the cache exceeds available GPU memory, older entries are either evicted to host memory or discarded. Subsequent requests that reference previously seen images or videos can retrieve their corresponding modal cache from GPU or host memory, bypassing redundant computation. This allows the system to skip recomputing the visual encoder and the prefill stage of the decoder for visual tokens, as shown in Figure 12(b). Furthermore, the system overlaps encoder computation for new requests with modal cache loading for prior requests, as illustrated in Figure 12(c), improving pipeline efficiency.

5.3 Modal Cache Compression

Multimodal models convert visual inputs into a sequence of tokens that interact with language components. For instance, BLIP-2 generates 32 visual tokens, while LLaVA produces up to 576, reflecting a higher granularity of visual information. A larger number of visual tokens increases the size of the modal cache, posing challenges for storage and loading performance. As shown in Figure 12(c), cache loading time can sometimes exceed encoder inference time.

To address this, EEVEE's controller prunes redundant entries in the modal cache by analyzing token-wise attention scores. Tokens with consistently low or sparse attention are

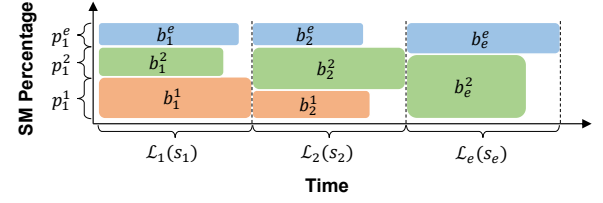


Figure 13. Notation definitions in the problem formulation of BLIP for VQA. If $b_1^1 = b_2^2 = b_e^e \neq 0$, the other $b_m^n = 0$, $p_1^1 = p_2^2 = p_e^e = 100$, and the other $p_m^n = 0$, the scheduling strategy of EEVEE is the same as that of the existing unimodal serving. Otherwise, it is a strategy of module multiplexing.

deemed to have minimal contribution to model outputs and are removed. This compression mechanism is modular and can be replaced with other strategies [4, 24, 49]. We refer to the compressed version as the *critical modal cache*. As shown in Figure 12(d), the full modal cache is stored in host memory due to its size, while the smaller critical modal cache is prioritized for GPU memory to support fast access. During inference, the controller dynamically decides whether to load the full or critical modal cache based on the number of eligible requests and system state. When GPU resources are tight or request volume is high, the system prefers the critical modal cache; otherwise, it retrieves the full cache to maximize accuracy and completeness.

6 Synergistic Scheduling Algorithm

Unlike previous spatio-temporal GPU sharing approaches for multiple models that only compete for resources [10, 17], the modules of a multimodal model process different modalities and must cooperate to fulfill requests. This motivates the design of a new scheduling algorithm that considers the synergistic nature of these modules.

6.1 Problem Formulation

The set of modules in a multimodal model is denoted as $\mathcal{M}$. Each inference of a multimodal model consists of multiple stages, with each stage corresponding to a module. Different stages share the GPU temporally. If the output of module m serves as the input for module n ($m, n \in \mathcal{M}$), then the stage of m precedes that of n . The egress module (e.g., the text decoder in BLIP) is represented by $e \in \mathcal{M}$.

As discussed in § 3, not only temporal multiplexing, but also spatial multiplexing of modules within a stage can benefit the inference of a multimodal model. Thus, during the execution of module $m \in \mathcal{M}$, multiple modules may run concurrently. During this stage, the batch size of module m is denoted as b_m^m , and its SM percentage as p_m^m . The batch size of module $n \in \mathcal{M} \setminus m$ is denoted as b_m^n , and its SM percentage is p_m^n . In each batch, the total batch size of module n is given by $b^n = \sum_{m \in \mathcal{M}} b_m^n$. The scheduling strategy of the serving system is denoted as $s = \{s_m\}_{m \in \mathcal{M}}$, where $s_m = \{(b_m^n, p_m^n)\}_{n \in \mathcal{M}}$.

Figure 13 shows an example of the problem formulation. Considering the characteristics of multimodal model serving, the strategy should adhere to the following three constraints:

First, at any times, the sum of the SM percentages of all modules $\mathcal{M}$ in a GPU should fully utilize, but not exceed, the GPU's maximum compute resource¹, i.e.,

$$\sum_{n \in \mathcal{M}} p_m^n = 100, \forall m \in \mathcal{M}. \quad (1)$$

Second, the computation of different modalities is inter-dependent because one module's input can be the output of another. Specifically, the batch size of each module is an integer multiple of the batch size of the egress module, i.e.,

$$b^n = k^n b^e, \forall n \in \mathcal{M}, \quad (2)$$

in which k^n is the batch size multiplier for module n . The setting is determined by two factors: 1) **Model Structure**: For instance, in BLIP for NVLR (Figure 2b), the image encoder's batch size is twice that of the text decoder. 2) **User Pattern**: For example, in the VQA task, each image corresponds to an average of three questions (**Finding 4**), making the image encoder's batch size one-third of the text decoder's.

Third, given a strategy s_m during the stage of module m , we denote the latency of module $n \in \mathcal{M}$ as $\mathcal{L}_n(s_m)$, which can be obtained from the offline profile. The latency of the stage of module m is denoted by $\mathcal{L}(s_m) = \max_{n \in \mathcal{M}} \mathcal{L}_n(s_m)$. Given the latency constraint of the multimodal model as Δ , the latency of a batch should satisfy

$$\mathcal{L}(s) = \sum_{m \in \mathcal{M}} \mathcal{L}(s_m) \leq \Delta. \quad (3)$$

For a strategy s^* to maximize the throughput of the multimodal serving system, we formulate the problem as follows

$$\begin{aligned} s^* &= \arg \max \mathcal{T}(s) \\ \text{s.t. } &(1), (2), (3). \end{aligned} \quad (4)$$

Problem Hardness. The strategy has a huge search space, specifically $B_{(|\mathcal{M}|-1)}^{99+|\mathcal{M}|}$ for the maximum batch size B and the set of modules $\mathcal{M}$. For example, if B is 32 for BLIP for VQA, the search space size is about 4×10^{12} . Moreover, $\mathcal{L}_n(s_m)$ is a black box, and offline profiling for all possible $\mathcal{L}_n(s_m)$ will be expensive. This is because the SM percentage is set as an environment variable before CUDA initialization

¹We set total active thread percentage to 100% rather than over-provisioning SMs [37], as we can reasonably estimate the workload of each module offline and allocate SMs accordingly. We also experimented with SM over-provisioning in the early stages, but observed increased resource contention between modules, leading to degraded overall performance. Thus, based on these empirical results, we chose to restrict total SM allocation to 100%, which provided more stable and predictable scheduling behavior.

Algorithm 1: Overall Workflow

Input: The set of modules $\mathcal{M}$, offline profile for all modules $\mathcal{L}_m(\cdot)$, latency constraint Δ

Output: The optimal resource scheduling strategy s^*

```

1 Init  $s = \{s_m\}_{m \in \mathcal{M}}$  where  $s_m = \{(b_m^n, p_m^n)\}_{n \in \mathcal{M}}$  as the
  best strategy by executing  $\mathcal{M}$  sequentially with a
  uniform batch size that meets constraints (1), (2), (3)
2 while true do
3   Let  $s^* = s$ 
4   for  $n \in \mathcal{M}$  do
5     // Profile the negative effect on the latency
6     // for new possible strategies
7     for  $m \in \mathcal{M}$  do
8       Let  $\beta_m^n = b_m^n + k^n$ 
9       Let  $\beta_m^r = b_m^r, \forall r \in \mathcal{M} \setminus n$ 
10      Let  $s'_m = \{(\beta_m^r, \rho_m^r)\}_{r \in \mathcal{M}}$  where  $s'_m$  meets
        constraint (1) and  $s'_m = \arg \min_{s'_m} \mathcal{L}(s'_m)$ 
11      // Accommodate  $n$ 's batch increment with the
        minimum negative effect on the latency
12      Let  $m = \arg \min_{m \in \mathcal{M}} (\mathcal{L}(s'_m) - \mathcal{L}(s_m))$ 
13      Update  $s'_m$  in  $s$ 
14   if constraint (3) is not met for  $s$  then break
15 return  $s^*$ 

```

and cannot be changed until the end of the process. Each profiling must restart the process, resulting in high downtime (2-15s) due to the framework startup cost [41].

6.2 Algorithm Design

We propose a greedy algorithm to mitigate the large search space and reduce the search cost for scheduling. The main idea is to incrementally increase the batch size of each modality, starting from a small value, and place the increment where it will have the minimum negative impact on batch latency, all while satisfying the three specified constraints. The complete procedure is detailed in **Algorithm 1**.

Coupled Module Initialization. To quickly identify an optimal scheduling strategy, it is crucial to start the process in a favorable state (line 1). Instead of random initialization, we initialize the state using the best strategy by treating the model as a monolithic unit (i.e., executing each module sequentially with a uniform batch size) that satisfies all three constraints: (1), (2), and (3). If the batch size increases without adjusting the SM allocation among modules, the strategy would violate one or more constraints.

Minimum Negative Increment. To increase throughput while adhering to the constraints, the algorithm decouples module scheduling and allows spatial GPU multiplexing between modules. The process is iterative and runs in several rounds. In each round, before increasing the batch size of a module (denoted as k^n for module $n \in \mathcal{M}$), we attempt to

place the increment in the stage of each module and reallocate SMs from other modules to the target module within the same stage. The impact of this placement can be estimated using a linear model-based prediction [5, 10]. A heuristic search, discussed later, determines a new SM percentage for the stage (lines 4-8). After simulating the result, the increment is placed in the stage with the minimum increase in latency (lines 9 and 10). The algorithm terminates when the batch size can no longer be increased (line 11).

Balanced SM Allocation. To minimize the profile cost during the search, the algorithm uses a heuristic design to determine the new percentages of SM allocation (line 8) instead of an exhaustive search. The latency of a stage is determined by the module with the longest execution time. Thus, the optimal SM allocation aims to equalize the latency of all modules within a stage. To achieve this, the algorithm transfers SMs from the module with the shortest latency to the module with the longest latency. For example, as illustrated in Figure 13, when placing Module 1 in the stage of the egress module, Module 2 has the shortest latency in the stage. Consequently, the algorithm transfers some SMs from Module 2 to Module 1. This process continues until the latency of all modules within the stage is approximately equal.

6.3 Adapting to Dynamic Workloads

Our offline scheduling algorithm is designed to compute optimal strategies for different combinations of GPU, model, and SLO requirements. For each configuration, it determines the maximum sustainable request throughput (i.e., max capacity) that satisfies SLO. If the incoming request rate exceeds this max capacity, the system responds by allocating additional GPUs and switching to the corresponding precomputed strategy. It eliminates the need to re-run the scheduler at runtime and ensures efficient adaptation to dynamic workloads.

For efficient switching, EEVEE employs a standard active-standby switching mechanism, similar to other MPS systems [14]. When the request pattern drifts beyond a preset threshold, the system launches a standby MPS instance with the closest pre-profile strategy. Both instances run concurrently during transition, and traffic is atomically redirected to the new one. Since model weights are already resident in memory, switching only updates scheduler metadata and completes within a few milliseconds. The threshold is computed using a sliding window over recent distribution. If no close match is found, EEVEE falls back to a conservative default strategy to ensure robustness.

6.4 GPU Scaling

To accelerate the inference of large modules (such as the LLM in an MLLM), EEVEE incorporates parallelism in a multi-GPU setting. As illustrated in Figure 14, the workflow consists of three main steps. First, the modules are classified into two categories: those that benefit from intra-operator model parallelism within GPUs and those that do not. For the latter,

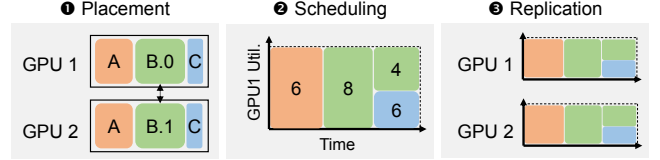


Figure 14. An example of EEVEE for serving a multi-modal model with three modules (i.e., A, B, and C) in two GPUs.

EEVEE employs a replication approach for data parallelism, where each GPU contains a replica of the module. Modules in the former category utilize intra-operator model parallelism to optimize performance. Second, EEVEE selects a GPU and applies **Algorithm 1** to determine its scheduling strategy, with two key modifications: (1) the latency of a module n in the first class, $\mathcal{L}_n(s_m)$, reflects the latency of one of its partitions after intra-operator parallelism, rather than the entire module; and (2) the batch size for modules in the second class is multiplied by the number of GPUs. Finally, the determined strategy is replicated across all GPUs to ensure consistent and efficient execution.

6.5 Analysis

The complexity of the algorithm is $100B|\mathcal{M}|^2$, far smaller than the problem’s search space of $B \binom{99+|\mathcal{M}|}{|\mathcal{M}|-1}^{|\mathcal{M}|}$. $|\mathcal{M}|^2$ denotes the process of placing the batch increment of every module in every stage, as shown in lines 4 and 5. 100 denotes the maximum possible number of SM percentages allocated to the module for each placement. This reduction in search space has a significant benefit in terms of convergence speed and performance improvement. A detailed analysis refers to Appendix A. The results in § 8 show that the proposed algorithm achieves 30% better performance than traditional heuristic algorithms with the same number of search steps.

The algorithm naturally generalizes to models with multiple modalities or nonlinear module dependencies, as the system represents a multimodal model as a directed acyclic graph in § 6.1. The only added challenge for more modalities lies in the expanded search space, but it remains tractable in practice due to the bounded number of modules per model for the existing multimodal models.

7 Implementation

We develop a prototype of EEVEE with approximately 5,000 lines of Python, utilizing vLLM [29] as the inference engine for the backend. For each specific module of the multimodal model, we launch a vLLM process configured with a distinct batch size and SM allocation setting. To allocate GPU resources to the modules, we use NVIDIA MPS [37]. While module multiplexing is demonstrated with MPS in our prototype, it can also be applied to MIG, which will be explored in future work. Within the same server, we use shared memory for inter-module communication. For each GPU, a

controller process manages the shared GPU memory space, allowing module processes and the scheduler to access it via the CUDA IPC handler. Additionally, to reduce memory usage caused by multiple processes using the same module, the controller stores a read-only copy of each module's parameters in the GPU's shared memory. The controller tags each module output to be cached using 64-bit hash. Entries $\langle \text{hash}, \text{tensor}, \text{last-access} \rangle$ are stored in per-module hashmaps. A global LRU queue manages eviction when GPU memory usage exceeds its quota. Stale tensors are either spilled to host memory or freed by a background thread overlapping with kernel launches, ensuring that compute operations are never stalled. To assess GPU performance, we utilize NVIDIA Nsight [38], a professional performance analysis tool developed by NVIDIA. In our work, we use the "SMs Active" metric in Nsight to measure GPU utilization.

8 Experiment

8.1 Experimental Setup

Workloads. We evaluate five multimodal models: CLIP [44], BLIP [32], BLIP-2 [31], LLaVA-1.5 [35], and InternVL2.5-8B [8]. Requests are generated from three datasets—VQA2 [20], NVLR2 [48], and COCO [34]—and support VQA, NVLR, and image-text matching (ITM). Workload request arrival patterns are modeled using Poisson distributions and sampled from traces in Azure [42], to simulate production-like workload arrival patterns and characteristics. For images with multiple questions, a query on an image is initiated after the previous query on the image has been completed. The modal cache is compressed at a 30% ratio.

Node setup. We evaluate EEVEE on two server configurations to test its scalability and generality across different hardware tiers. The high-end server is equipped with an Intel Xeon Gold 6230R CPU (104 cores), 503 GB of memory, and 8 NVIDIA A100 40GB GPUs, representing a modern data-center-grade setup for large-scale inference workloads. The low-end server is equipped with an Intel Xeon E5-2698 v4 CPU (80 cores), 377 GB of memory, and 8 NVIDIA GeForce RTX 3090 GPUs, representing a more affordable commodity hardware configuration. All GPUs are connected to their host CPUs via PCIe 3.0, offering a peak bandwidth of 32 GB/s. Both servers run Ubuntu 20.04.5, Docker 23.0.1, CUDA 11.8, Python 3.8, and PyTorch 2.0.1.

Baselines. We compare EEVEE against two serving systems originally designed for unimodal models. (1) Triton [39] is a widely used serving framework developed by NVIDIA. It treats the multimodal model as a monolithic graph, executing all modules sequentially with a uniform batch size. This approach represents the dominant paradigm adopted by most existing serving systems when extended to support multimodal models. (2) Gpulet [10] is a system that enables spatial and temporal GPU sharing across multiple concurrent models. To support multimodal inference in Gpulet, we

treat each module of a multimodal model as an independent model and enforce a fixed, shared batch size across all modules. For a fair comparison, all baselines are integrated with the same underlying inference engine, vLLM [29].

Metrics. First, we use a widely used throughput metric, *max capacity*, which is defined as the maximum request load (in queries per second) that a serving system can sustain while meeting certain latency targets (including average SLO and 99th percentile (P99) SLO). Second, we use *average and P99 latency* to measure the delay between the time a user submits a request and the time the request is completed. Third, we use the *percentage of active SMs* to measure the average GPU utilisation of a serving system within a time slot. Evaluation results are based on the average of three random trials to account for unpredictable performance fluctuations.

8.2 Capacity

We evaluate the performance of EEVEE, Triton, and Gpulet on BLIP for VQA and NLVR tasks under the Poisson and Azure workloads. For VQA, the SLO is defined as 4 $\times$, 6 $\times$, and 8 $\times$ the inference latency of BLIP with a batch size of 2. For NLVR, the SLO is defined as 4 $\times$, 6 $\times$, and 8 $\times$ the inference latency of BLIP with a batch size of 1. Note that this refers only to how the SLO is defined—not the batch size used during execution. At runtime, batch sizes are determined dynamically by the request arrival rate, the SLO target, and the multiplexing strategy.

As shown in Figure 15 and Figure 16, EEVEE outperforms Triton and Gpulet in most cases. For instance, on a 3090 server, EEVEE improves the maximum capacity by 157% on average for the Azure Workload compared to the best baseline. For Triton, only temporal multiplexing leads to resource underutilization. For Gpulet, there are scenarios where certain modules require substantial computational resources, making exclusive utilization a more effective option. In contrast, EEVEE's integration of both temporal and spatial multiplexing overcomes this inefficiency. It not only avoids inefficient resource allocation for modules with minimal computational demands but also ensures that modules with high computational needs have access to the necessary resources. At the same time, the preprocessing and request decoupling of the module multiplexing improves GPU utilisation.

With a more stringent SLO, EEVEE exhibits performance 2.54 times higher than Triton and 1.9 times higher than Gpulet. This is because Triton, in this case, has a limited batch size, causing lower utilization. This suggests that EEVEE has greater potential for optimization. A reduction in the SLO threshold enables Triton to expand the batch size, thereby improving utilization. Additionally, for the Poisson Workload, the capacity is greater than that of the Azure Workload. Given that the Azure Workload exhibits a higher coefficient of variation, the system experiences more burst requests, which may temporarily exceed the system's capacity, leading to a significant amount of queuing delay.

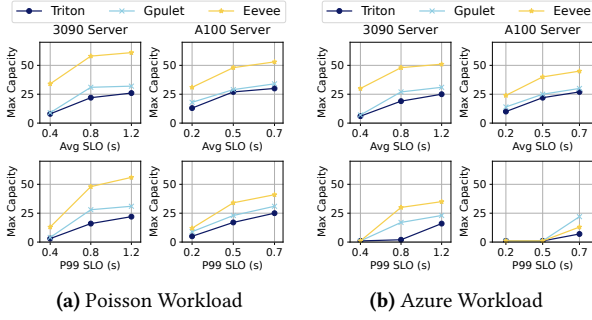


Figure 15. Capacity (in queries per second) of BLIP for VQA.

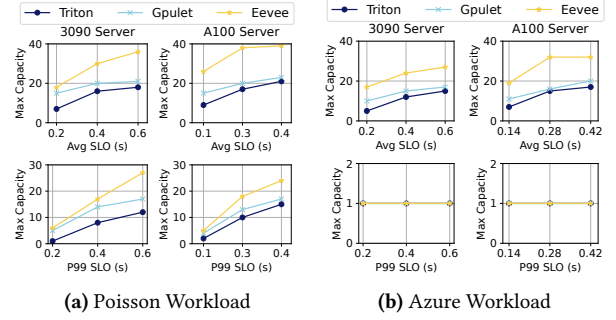


Figure 16. Capacity (in queries per second) of BLIP for NVLR.

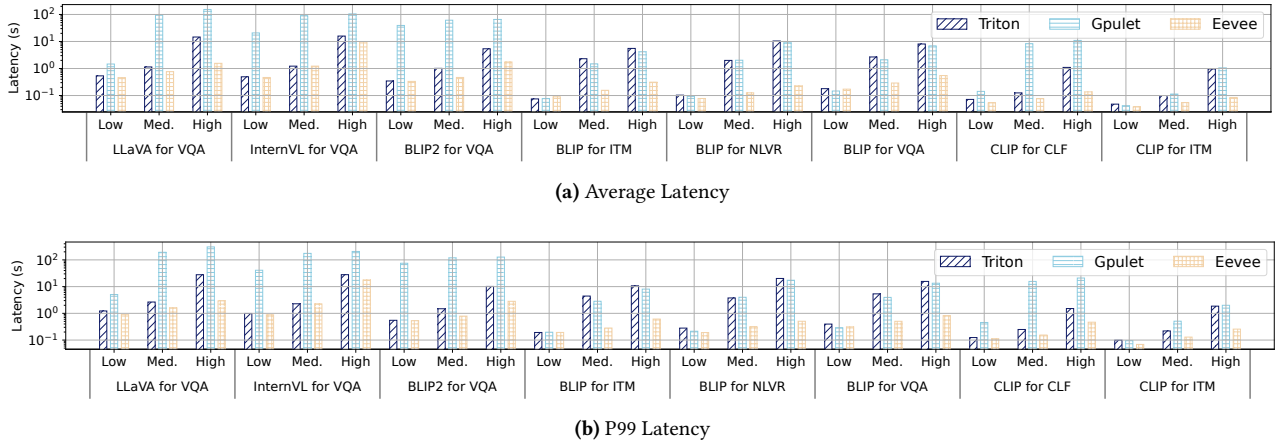


Figure 17. Latency of five multimodal models at low, medium, high load with Triton, Gpulet, and EEEVE.

8.3 Latency

We evaluate EEEVE, Triton, and Gpulet on five multimodal models under the Poisson workload on an A100 server. The SLO is defined as $4\times$ the inference latency for each task with a batch size of 2. The load levels are defined as $\frac{1}{4}$, $\frac{3}{5}$, and $\frac{9}{10}$ of EEEVE’s peak throughput, corresponding to low, medium, and high loads, respectively. As shown in Figure 17, EEEVE reduces both average latency and P99 latency for both MTMs and MLLMs across various workloads compared to Triton and Gpulet. For instance, for LLaVA, EEEVE decreases the average latency by about 90% compared to Triton under high workload conditions. As the workload increases, the latency of all systems increases. However, the performance improvement of EEEVE becomes more significant with higher workloads. This is because EEEVE can fully utilize GPU resources, mitigating the latency increase under higher workloads.

8.4 Micro-benchmark

GPU Utilization. As shown in Figure 18, EEEVE demonstrates significantly higher GPU utilization when serving

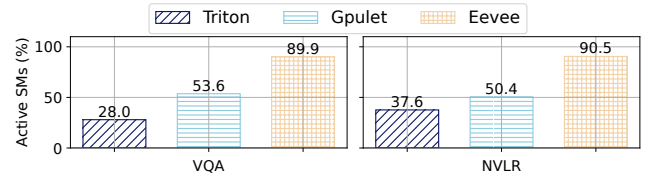


Figure 18. Average percentage of active SMs for BLIP.

multimodal models compared to the baselines. In particular, EEEVE achieves, on average, nearly 90% active SMs in BLIP for VQA and NVLR. As shown in Figure 4, there is significant time with low GPU utilization in Triton. While Gpulet supports spatio-temporal GPU multiplexing between modules, its average SM percentage is higher than Triton’s. However, Gpulet does not account for stage-level parallelism and request-aware reusing, which results in lower GPU utilization compared to EEEVE.

Scheduling Algorithms. Figure 20 shows the performance of the synergistic scheduling algorithm in EEEVE compared

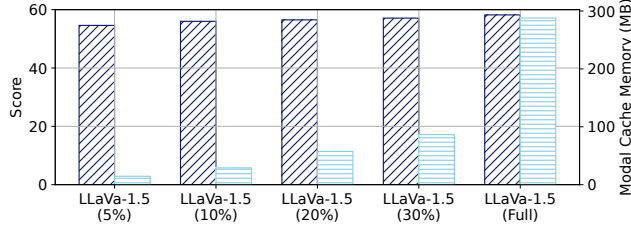


Figure 19. Modal cache size and score under different budgets for modal cache compression.

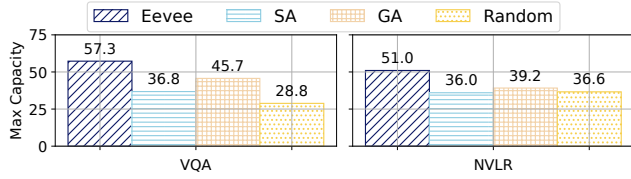


Figure 20. Comparison of four scheduling algorithms (SA: Simulated Annealing and GA: Genetic Algorithm) for BLIP.

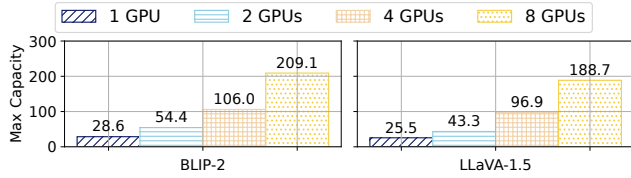
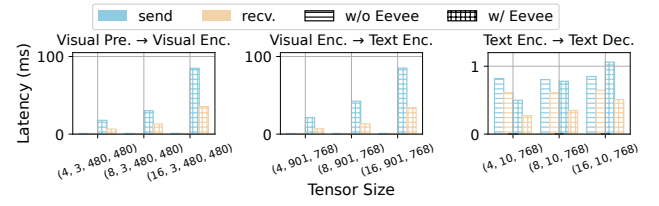


Figure 21. Capacity (in queries per second) when there are different numbers of GPUs for BLIP-2 and LLaVA.

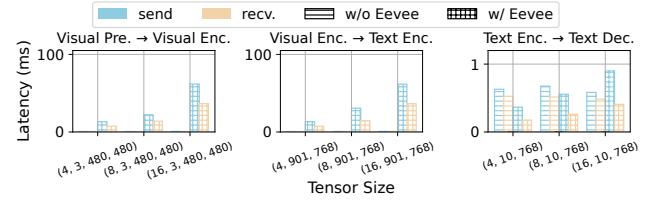
to three traditional heuristic algorithms. The heuristic algorithms include a simulated annealing algorithm, a genetic algorithm, and a greedy algorithm that randomly selects solutions to solve the problem in Equation 4. To ensure a fair comparison, all algorithms are run with the same number of search steps, and the hyperparameters are carefully chosen. The results demonstrate that the proposed algorithm achieves a performance improvement of up to 30.10% compared to the most effective baseline. This is attributed to the three designs of the proposed algorithm (including coupled module initialization, minimum negative increment, and balanced SM allocation), which reduces the search space.

Modal Cache Management. Figure 19 shows the size of the modal cache and the score after different compression budgets for LLaVA. As the budget decreases, i.e. more modal cache is compressed, the size of the modal cache decreases significantly, while the score decreases only slightly. This makes it easier to store and load the modal cache in EEEVEE.

Multi-GPU Environment. Figure 21 shows the maximum capacity of EEEVEE in a multi-GPU system with varying numbers of A100 GPUs. As the number of GPUs in EEEVEE increases, throughput scales almost linearly, demonstrating



(a) 3090 Server



(b) A100 Server

Figure 22. Data transfer latency between modules.

the system’s ability to efficiently handle growing workloads. This performance improvement is due to EEEVEE’s integration of both model and data parallelism in a multi-GPU setting, as discussed in § 6.4. Model parallelism divides the module across multiple GPUs, allowing simultaneous processing of different parts, while data parallelism distributes data batches across GPUs for concurrent processing. Together, these strategies ensure efficient use of resources and scalability as more GPUs are added to the system.

Multiplexing Overhead. Figure 22 shows the data transfer between the processes of modules caused by the module multiplexing, as discussed in § 4. The results demonstrate that using shared memory for inter-module communication, as outlined in § 7, significantly reduces the transfer latency between modules, especially when the size of the embeddings to be transferred is large.

Ablation Study. We conduct an ablation study on the 3090 server using BLIP for VQA under a Poisson-simulated workload. We evaluate the impact of each component of our design in isolation. Specifically, stage-level parallelism, module-level scheduling, and request-aware reuse improve the system’s request-handling capacity—measured under average SLO constraints—by 146%, 152%, and 116%, respectively.

8.5 Generalizability to More Modalities

We select vision-language models for evaluation based on two considerations. First, visual understanding represents a mature and widely adopted class of multimodal tasks, supported by strong benchmarks, well-established model architectures, and a broad range of real-world applications. These characteristics make vision-language tasks a practical and impactful testbed for analyzing multimodal serving

behavior. Second, many modern multimodal models process non-visual modalities using transformer-based encoder architectures that are structurally similar to those used in the visual domain. For instance, ImageBind [21, 51] encodes modalities such as audio, depth, IMU, and thermal signals using ViT-like transformer blocks that operate on tokenized inputs. Besides, many visual-language-action models [27] for embodied AI are fine-tuned from vision-language models. This architectural consistency supports the generalizability of our design and findings beyond vision-centric tasks. In addition, we provide a preliminary evaluation indicating EEVEE’s potential on diffusion-based modules in Appendix B.

9 Related Work

9.1 Efficient Unimodal Model Inference

Batching. Many serving systems utilize batching techniques to enhance inference efficiency, including TF-serving [40], Clipper [12], TurboTransformers [18], and Triton [39]. They typically employ traditional batching strategies, which rely on batch time windows and a maximum allowed batch size, treating the model as an indivisible whole. A closely related approach is DVABatch [13], which optimizes batching by accounting for input, operator, and load diversity within a model. DVABatch enables operators within a model to execute at different batch sizes based on their computational complexity. However, while effective for unimodal models, DVABatch does not adapt its batching strategies or GPU multiplexing to the intricate internal structures and interdependencies characteristic of modules in a multimodal model.

GPU Multiplexing. To improve GPU utilization, some approaches load multiple models into GPU memory to share a single GPU [7, 15, 46, 47, 55]. For example, Choi *et al.* propose a spatio-temporal multiplexing system for multi-model ML inference servers [10], while Duan *et al.* introduce MuxServe, a spatio-temporal multiplexing system specifically designed for efficient serving of multiple LLMs [17]. However, these spatio-temporal GPU sharing techniques are limited to resource competition among independent models and fail to adapt to the synergistic interactions between different modules within a multimodal model, as discussed in § 6.

9.2 Efficient Multimodal Model Inference

Algorithm-level Design. Many works have been conducted to optimise the algorithms of multimodal models to accelerate their inference [26]. We summarize them as follows. 1) Quantization: Shi *et al.* propose a universal multimodal pruning framework UPop for compressing vision-language Transformers [45]. 2) Token merging/pruning: Cao *et al.* propose a multimodal alignment-guided dynamic token pruning approach [24]. 3) Knowledge distillation: Jin *et al.* investigate the knowledge transfer from a large multimodal model to a smaller one [25]. These works are orthogonal to EEVEE and can be combined together to enable better performance.

System-level Design. Several works improve the efficiency of multimodal model training through system-level design [50, 57]. For example, Huang *et al.* propose an efficient training system called DISTMM for multimodal models [23]. It exploits the heterogeneity of modules by applying different distributed parallelism strategies for each module. Feng *et al.* propose Optimus, a distributed MLLM training system that enables the scheduling of encoder computation within LLM bubbles to achieve performant 3D parallelism [19]. Similar to these works, we also observe the heterogeneity of the modules in a multimodal model. However, most of them aim to mitigate the negative influence of module heterogeneity on the parallelism paradigms of distributed training for multimodal models. Meta [30] evaluates the performance of its multimodal models and analyzes the effectiveness of some system-level and workload-specific optimizations applied to them. ModServe [43], a concurrent work, also targets efficient multimodal serving but adopts a different strategy. ModServe places different modules on separate GPUs, using a method conceptually similar to prefill-decode disaggregation. In contrast, our method focuses on intra-GPU scheduling and SM-level optimization. Both approaches are orthogonal, addressing complementary aspects of the system design, and could be integrated in future work.

10 Conclusion

We present EEVEE, an efficient multimodal model serving system that adopts a module multiplexing approach. EEVEE performs fine-grained scheduling across modality-specific modules by independently managing their computation and resource allocation. This design significantly improves GPU utilization and system throughput. Experimental results show that EEVEE improves throughput by 157%, reduces latency by 90%, and achieves up to 90% GPU utilization under both Poisson-simulated and real-world workloads from Azure. Future work focuses on extending support to a broader range of multimodal models, such as vision-language-action (VLA) models for embodied AI and robotics applications.

Acknowledgments

This research was supported by fundings from the Hong Kong RGC General Research Fund (152169/22E, 152228/23E, 162161/24E, 162116/25E), Research Impact Fund (No. R5060-19, No. R5011-23), Collaborative Research Fund (No. C1042-23GF), NSFC/RGC Collaborative Research Scheme (Grant No. 62461160332 & CRS_HKUST602/24), Areas of Excellence Scheme (AoE/E-601/22-R), the InnoHK (HKGAI), and the National Natural Science Foundation of China (No. 62471383). Song Guo and Xiaowei Shen are the corresponding authors. We thank all anonymous reviewers and our shepherd Rong Chen, who helped improve the paper. We also thank MetaX Technology for supporting this research.

References

- [1] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C. Lawrence Zitnick, and Devi Parikh. 2015. VQA: Visual Question Answering. In *International Conference on Computer Vision*.
- [2] Jiuhai Chen, Zhiyang Xu, Xichen Pan, Shusheng Yang, Can Qin, An Yan, Honglu Zhou, Zeyuan Chen, Tianyi Zhou, Silvio Savarese, Le Xue, Caiming Xiong, and Ran Xu. 2025. BLIP3o-NEXT: A Next-Generation Multimodal Foundation Model. <https://jiuhaichen.github.io/BLIP3o-NEXT.github.io/>
- [3] Kevin Chen, Junshen K. Chen, Jo Chuang, Marynel Vázquez, and Silvio Savarese. 2021. Topological Planning with Transformers for Vision-and-Language Navigation. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11271–11281.
- [4] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. In *European Conference on Computer Vision*.
- [5] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: QoS Awareness and Increased Utilization for Non-Preemptive Accelerators in Warehouse Scale Computers. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*. 681–696.
- [6] Wenhui Chen, Ming-Wei Chang, Eva Schlinger, William Yang Wang, and William W. Cohen. 2021. Open Question Answering over Tables and Text. In *International Conference on Learning Representations*.
- [7] Wenyan Chen, Zizhao Mo, Huanle Xu, Kejiang Ye, and Chengzhong Xu. 2023. Interference-aware Multiplexing for Deep Learning in GPU Clusters: A Middleware Approach. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [8] Zhe Chen, Jiannan Wu, Wenhui Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. 2024. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 24185–24198.
- [9] Yang Cheng, Dan Li, Zhiyuan Guo, Binyao Jiang, Jiaxin Lin, Xi Fan, Jinkun Geng, Xinyi Yu, Wei Bai, Lei Qu, Ran Shu, Peng Cheng, Yongqiang Xiong, and Jianping Wu. 2019. DLBooster: Boosting End-to-End Deep Learning Workflows with Offloading Data Preprocessing Pipelines. In *International Conference on Parallel Processing*.
- [10] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing. In *USENIX Annual Technical Conference*. 199–216.
- [11] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022. Scaling Instruction-Finetuned Language Models. [arXiv:2210.11416 \[cs.LG\]](https://arxiv.org/abs/2210.11416) <https://arxiv.org/abs/2210.11416>
- [12] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *USENIX Symposium on Networked Systems Design and Implementation*. 613–627.
- [13] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. 2022. DVABatch: Diversity-aware Multi-Entry Multi-Exit Batching for Efficient Processing of DNN Services on GPUs. In *USENIX Annual Technical Conference*. 183–198.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4171–4186.
- [15] Aditya Dhakal, Sameer G Kulkarni, and K. K. Ramakrishnan. 2020. GSLICE: controlled spatial sharing of GPUs for a scalable inference platform. In *ACM Symposium on Cloud Computing*. 492–506.
- [16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR* (2021).
- [17] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. 2024. MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving. In *Forty-first International Conference on Machine Learning*.
- [18] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. TurboTransformers: An Efficient GPU Serving System for Transformer Models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 389–402.
- [19] Weiqi Feng, Yangrui Chen, Shaoyu Wang, Yanghua Peng, Haibin Lin, and Minlan Yu. 2024. Optimus: Accelerating Large-Scale Multi-Modal LLM Training by Bubble Exploitation. [arXiv:2408.03505 \[cs.CL\]](https://arxiv.org/abs/2408.03505) <https://arxiv.org/abs/2408.03505>
- [20] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. 2017. Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [21] Jiaming Han, Renrui Zhang, Wenqi Shao, Peng Gao, Peng Xu, Han Xiao, Kaipeng Zhang, Chris Liu, Song Wen, Ziyu Guo, Xudong Lu, Shuai Ren, Yafei Wen, Xiaoxin Chen, Xiangyu Yue, Hongsheng Li, and Yu Qiao. 2023. ImageBind-LLM: Multi-modality Instruction Tuning. [arXiv:2309.03905 \[cs.MM\]](https://arxiv.org/abs/2309.03905) <https://arxiv.org/abs/2309.03905>
- [22] Yicong Hong, Qi Wu, Yuankai Qi, Cristian Rodriguez-Opazo, and Stephen Gould. 2021. VLNBERT: A Recurrent Vision-and-Language BERT for Navigation. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 1643–1653.
- [23] Jun Huang, Zhen Zhang, Shuai Zheng, Feng Qin, and Yida Wang. 2024. DISTMM: Accelerating Distributed Multimodal Model Training. In *Networked Systems Design and Implementation*. 1157–1171.
- [24] Cao Jianjian, Ye Peng, Li Shengze, Yu Chong, Tang Yansong, Lu Jiwen, and Chen Tao. 2024. MADTP: Multimodal Alignment-Guided Dynamic Token Pruning for Accelerating Vision-Language Transformer. *IEEE Conference on Computer Vision and Pattern Recognition* (2024).
- [25] Woojeong Jin, Maziar Sanjabi, Shaoliang Nie, Liang Tan, Xiang Ren, and Hamed Firooz. 2021. MSD: Saliency-aware Knowledge Distillation for Multimodal Understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 3557–3569.
- [26] Yizhang Jin, Jian Li, Yexin Liu, Tianjun Gu, Kai Wu, Zhengkai Jiang, Muyang He, Bo Zhao, Xin Tan, Zhenye Gan, Yabiao Wang, Chengjie Wang, and Lizhuang Ma. 2024. Efficient Multimodal Large Language Models: A Survey. [arXiv:2405.10739 \[cs.CV\]](https://arxiv.org/abs/2405.10739)
- [27] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. [arXiv preprint arXiv:2406.09246](https://arxiv.org/abs/2406.09246) (2024).
- [28] Wonjae Kim, Bokyung Son, and Ildoo Kim. 2021. ViLT: Vision-and-Language Transformer Without Convolution or Region Supervision. In *International Conference on Machine Learning*. 5583–5594.

- [29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [30] Yejin Lee, Anna Sun, Basil Hosmer, Bilge Acun, Can Balioglu, Changhan Wang, Charles David Hernandez, Christian Puhersch, Daniel Haziza, Driss Guessous, Francisco Massa, Jacob Kahn, Jeffrey Wan, Jeremy Reizenstein, Jiaqi Zhai, Joe Isaacson, Joel Schlosser, Juan Pino, Kaushik Ram Sadagopan, Leonid Shamis, Linjian Ma, Min-Jae Hwang, Mingda Chen, Mostafa Elhoushi, Pedro Rodriguez, Ram Pasunuru, Scott Yih, Sravya Popuri, Xing Liu, and Carole-Jean Wu. 2025. Characterizing and Efficiently Accelerating Multimodal Generation Model Inference. arXiv:2410.00215 [cs.LG] <https://arxiv.org/abs/2410.00215>
- [31] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. arXiv:2301.12597 [cs.CV]
- [32] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. 2022. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. arXiv:2201.12086 [cs.CV]
- [33] Yikuan Li, Shishir Rao, José Roberto Ayala Solares, Abdelaali Hassaine, Rema Ramakrishnan, Dexter Canoy, Yajie Zhu, Kazem Rahimi, and Gholamreza Salimi-Khorshidi. 2020. BEHRT: transformer for electronic health records. *Scientific reports* 10, 1 (2020), 7155.
- [34] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. 2015. Microsoft COCO: Common Objects in Context. arXiv:1405.0312 [cs.CV] <https://arxiv.org/abs/1405.0312>
- [35] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Neural Information Processing Systems*.
- [36] NVIDIA. 2025. NVIDIA Multi-Instance GPU (MIG). Retrieved March 15, 2025 from <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/index.html>
- [37] NVIDIA. 2025. NVIDIA Multi-Process Service (MPS). Retrieved March 15, 2025 from <https://docs.nvidia.com/deploy/mps/index.html>
- [38] NVIDIA. 2025. NVIDIA Nsight Systems. Retrieved March 15, 2025 from <https://developer.nvidia.com/nsight-systems>
- [39] NVIDIA. 2025. NVIDIA Triton Inference Server. Retrieved March 15, 2025 from <https://www.nvidia.com/en-us/ai-data-science/products/triton-inference-server/>
- [40] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139* (2017).
- [41] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc.
- [42] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*.
- [43] Haoran Qiu, Anish Biswas, Zihan Zhao, Jayashree Mohan, Alind Khare, Esha Choukse, Inigo Goiri, Zeyu Zhang, Haiying Shen, Chetan Bansal, Ramachandran Ramjee, and Rodrigo Fonseca. 2025. ModServe: Scalable and Resource-Efficient Large Multimodal Model Serving. arXiv:2502.00937 [cs.DC] <https://arxiv.org/abs/2502.00937>
- [44] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. arXiv:2103.00020 [cs.CV]
- [45] Dachuan Shi, Chaofan Tao, Ying Jin, Zhendong Yang, Chun Yuan, and Jiaqi Wang. 2023. UPop: Unified and Progressive Pruning for Compressing Vision-Language Transformers. In *Proceedings of the 40th International Conference on Machine Learning*. 31292–31311.
- [46] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. 2024. USHER: Holistic Interference Avoidance for Resource Optimized ML Inference. In *USENIX Symposium on Operating Systems Design and Implementation*. 947–964.
- [47] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 1075–1092.
- [48] Alane Suhr, Stephanie Zhou, Ally Zhang, Iris Zhang, Huajun Bai, and Yoav Artzi. 2019. A Corpus for Reasoning about Natural Language Grounded in Photographs. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 6418–6428.
- [49] Zhongwei Wan, Ziang Wu, Che Liu, Jinfa Huang, Zhihong Zhu, Peng Jin, Longyue Wang, and Li Yuan. 2024. LOOK-M: Look-Once Optimization in KV Cache for Efficient Multimodal Long-Context Inference. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 4065–4078.
- [50] Yujie Wang, Shenhan Zhu, Fangcheng Fu, Xupeng Miao, Jie Zhang, Juan Zhu, Fan Hong, Yong Li, and Bin Cui. 2024. Efficient Multi-Task Large Model Training via Data Heterogeneity-aware Model Management. arXiv:2409.03365 [cs.DC] <https://arxiv.org/abs/2409.03365>
- [51] Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2023. NExT-GPT: Any-to-Any Multimodal LLM.
- [52] Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. 2017. Video Question Answering via Gradually Refined Attention over Appearance and Motion. In *ACM Multimedia*.
- [53] Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. 2024. A Survey on Multimodal Large Language Models. arXiv:2306.13549 [cs.CV] <https://arxiv.org/abs/2306.13549>
- [54] Jinliang Yuan, Chen Yang, Dongqi Cai, Shihe Wang, Xin Yuan, Zeling Zhang, Xiang Li, Dingge Zhang, Hanzi Mei, Xianqing Jia, Shangguang Wang, and Mengwei Xu. 2024. Mobile Foundation Model as Firmware. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 279–295.
- [55] Bowen Zhang, Shuxin Li, and Zhuozhao Li. 2024. MIGER: Integrating Multi-Instance GPU and Multi-Process Service for Deep Learning Clusters. In *International Conference on Parallel Processing*. 504–513.
- [56] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuhui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL] <https://arxiv.org/abs/2205.01068>
- [57] Zili Zhang, Yinmin Zhong, Ranchen Ming, Hanpeng Hu, Jianjian Sun, Zheng Ge, Yibo Zhu, and Xin Jin. 2024. DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large Language Models. arXiv:2408.04275 [cs.DC] <https://arxiv.org/abs/2408.04275>
- [58] Hao Zhou, Wengang Zhou, Weizhen Qi, Junfu Pu, and Houqiang Li. 2021. Improving Sign Language Translation with Monolingual Data by Sign Back-Translation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 1316–1325.

A Convergence and Optimality Guarantees

We adopt the notation and constraints of § 6.1. We use $\mathcal{M}$ to index both modules and stages; the egress module is e . A scheduling strategy is $\mathbf{s} = \{s_m\}_{m \in \mathcal{M}}$, where $s_m = \{(b_m^n, p_m^n)\}_{n \in \mathcal{M}}$ denotes, for stage m , the batch assignments

b_m^n and SM percentages p_m^n for each module n . The effective batch of module n is $b^n = \sum_{m \in \mathcal{M}} b_m^n = k^n b^e$ with integer multipliers $k^n \geq 1$. Stage latency is $\mathcal{L}(s_m) = \max_{n \in \mathcal{M}} \mathcal{L}_n(s_m)$ and end-to-end latency is $\mathcal{L}(s) = \sum_{m \in \mathcal{M}} \mathcal{L}(s_m)$. The SLO requires $\mathcal{L}(s) \leq \Delta$. Throughput is $\mathcal{T}(s) = b^e / \mathcal{L}(s)$.

Algorithmic move set. **Algorithm 1** initializes from the best *coupled* (monolithic) feasible strategy and then iteratively: (i) picks a module n and proposes to increase its batch by one multiplier unit k^n ; (ii) tries placing this increment in each stage $m \in \mathcal{M}$; (iii) rebalances the stage's SM percentages to minimize the induced increase in $\mathcal{L}(s_m)$ ("Minimum Negative Increment"); and (iv) commits the least-harmful feasible placement *only if* it does not reduce $\mathcal{T}(s)$. The search stops when no such single-increment move exists.

Assumptions. **(A1) Finite discrete space.** For each module n , the admissible batch b^n lies on a finite grid determined by SLO. Let B be the global maximum egress batch in § 6.1. By the coupling constraint $b^n = k^n b^e$, we set $B^n = k^n B$ and restrict candidates to $\{0, k^n, 2k^n, \dots, B^n\}$. **(A2) Feasibility oracle.** For any candidate increment and within-stage SM reallocation, feasibility $\mathcal{L}(s) \leq \Delta$ is testable via the profiled latency model. **(A3) Monotone latency.** Increasing a module's batch weakly increases its own latency and does not decrease others' latency within the same stage; increasing a module's SM share weakly decreases its latency while (under fixed 100% SM per stage) weakly increasing the rest.

Theorem A.1 (Feasibility invariants). *Starting from the coupled feasible initialization, every accepted update preserves all constraints (per-stage SM budget, batch coupling, and SLO). Hence all intermediate and final strategies are feasible.*

Proof. Each candidate increment respects coupling by adding an integer multiple k^n ; the stage's SM percentages are rebalanced to satisfy the per-stage SM budget; the update is committed only if the recomputed latency obeys $\mathcal{L}(s) \leq \Delta$. Therefore, feasibility is an invariant by construction. $\square$

Theorem A.2 (Finite-time convergence and complexity). *Algorithm 1 terminates after at most $B - b_{\text{init}}^e \leq B$ accepted increments, where B is the global maximum egress batch (cf. Assumption A1) and b_{init}^e is the egress batch in the coupled initialization. Moreover, the algorithm performs at most*

$$O(|\mathcal{M}|^2 \cdot (B - b_{\text{init}}^e) \cdot P)$$

stage evaluations, where $P \leq 100$ upper-bounds the discrete SM reallocation trials per stage.

Proof. By the coupling constraint $b^n = k^n b^e$, any accepted update that adds one multiplier unit k^n to some b^n is equivalent to increasing the egress batch by one. Assumption (A1) restricts b^e to $\{0, 1, \dots, B\}$, hence at most $B - b_{\text{init}}^e$ accepted increments are possible, proving termination. For the complexity bound, in each accepted step the algorithm may examine $O(|\mathcal{M}|)$ choices of the module n , $O(|\mathcal{M}|)$ candidate

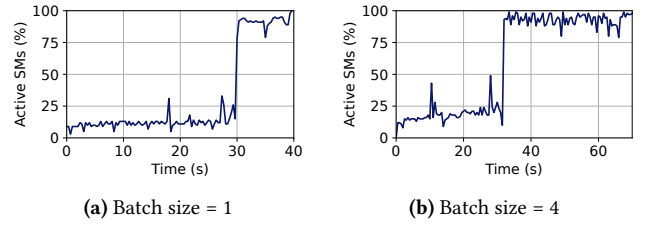


Figure 23. Percentage of active SMs for BLIP3o-NEXT-3B with varying batch size in RTX 3090 GPU.

stage placements m , and up to P SM reallocations, yielding $O(|\mathcal{M}|^2 \cdot P)$ stage evaluations per accepted step. Multiplying by the number of accepted steps $B - b_{\text{init}}^e$ gives the bound. $\square$

Theorem A.3 (Monotone improvement and 1-exchange optimality). *Let $\{s_t\}_{t \geq 0}$ denote the subsequence of strategies produced by Algorithm 1 immediately after each accepted update, with s_0 being the coupled initialization, and let s^* be the final returned strategy. Then: (i) for every $t \geq 0$, $\mathcal{T}(s_{t+1}) \geq \mathcal{T}(s_t)$; and (ii) s^* is 1-exchange optimal: there is no single feasible increment (increasing any b^n by k^n and rebalancing SMs within any one stage) that yields strictly higher throughput.*

Proof. (i) By design, the algorithm commits an update only if it does not reduce throughput, hence $\mathcal{T}(s_{t+1}) \geq \mathcal{T}(s_t)$ for all $t \geq 0$. (ii) Upon termination, every single-increment proposal either violates feasibility ($\mathcal{L}(\cdot) \leq \Delta$) or fails the non-decreasing-throughput acceptance test; therefore no strictly improving 1-exchange move exists, establishing 1-exchange optimality. $\square$

B Extending to Diffusion-based Modules

In § 3.1, we show that reusing unimodal serving systems for BLIP/BLIP-2-style multimodal models leads to pronounced GPU under-utilization. Here we extend the evaluation to a newer architecture BLIP3o-NEXT [2] whose output module is diffusion-based. It first uses an autoregressive (AR) module to produce intermediate features, and then conditions a diffusion decoder on these features to synthesize images.

As illustrated in Figure 23, we observe a clear asymmetry between the two phases of BLIP3o-NEXT: (i) the AR phase exhibits persistently low SM utilization and shows weak sensitivity to batch size; (ii) the diffusion phase is compute-dominated and its latency grows sharply with batch size. Our module multiplexing is expected to exploit this heterogeneity by overlapping a portion of diffusion denoising with the AR compute and rebalance per-stage SM shares to preserve the SLO. This scheduling will increase GPU utilization and shorten end-to-end latency. This experiment is intended to surface the headroom of module multiplexing in diffusion-augmented multimodal pipelines and to motivate dedicated diffusion-aware designs as future work.